Install
Usage summary
Pingable Node (Part 1)
Starting a node
Join a network
Getting your IP address
Sockets
Client and Server (Part 2)
TCP Server
TCP Client
Android Examples
ExoPlayer client
HttpURLConnection client
OkHttp client
Questions
Identities

Sockets

Encrypted P2P connections for your app or service.

This guide explains how to use the ZeroTier SDK Socket API. It is meant to be read linearly and progresses from beginner topics to advanced topics. We will start by creating a simple pingable node while skipping over most of the gritty details. Then we'll move on to a full client-server socket application where we will take the occasional tangent to learn more about how all of this works. Source code for the examples can be found here: libzt/examples. For API reference documentation see the sidebar to the left. To read more more about how ZeroTier works in general, see our Design Whitepaper. If you find an error on this page or you just need help getting something to work please open a GitHub issue.

Install

C/C++ Rust Python C# Java

```
brew install zerotier/tap/libzt
```

Alternatively, build from source: github.com/zerotier/libzt

INFO

For the purposes of this guide it is recommended that you first create an account and a private network to test on, but it is not strictly necessary if you're comfortable joining public networks or setting up your own network controller.

Usage summary

C/C++ Rust Python C# Java

```
#include <ZeroTierSockets.h>

// ...
zts_node_start();
zts_net_join(0x1234567890abcdef);
// ...
int s = zts_socket(ZTS_AF_INET, ZTS_SOCK_STREAM, 0);
zts_connect(s, in4, adddrlen);
// ...
zts_node_stop();
```

Pingable Node (Part 1)

To start things off we will:

- Set up a node
- Join it to your network
- Assign it a globally-available static IPv4 and/or IPv6 address
- Ping it

We'll be skipping over a lot of details in this first example, but we'll cover those in the next example. It's also worth noting that in this first example we will be using the sequential API for simplicity but there is an event notification system that can be used as well.

Starting a node

Starting a node is easy and if you don't provide a configuration ZeroTier will generate a new `identity` automatically, but don't worry about that yet:

C/C++ Rust Python C# Java

```
// No "node object" creation necessary in C
zts_node_start();
```

Before we can join any networks or use sockets we need to know when the node has successfully come online. We will use a rather crude wait loop but as mentioned before, there are event-based methods for accomplishing this that we will discuss later:

C/C++ Rust Python C# Java

```
while (! zts_node_is_online()) {
  zts_util_delay(1000);
}
```

Once we've broken from this loop we can be confident that the node has a valid identity and has made contact with at least one root server. At this point let's print our node's identity. This is the `public short form` of your identity that you can share with people, we will discuss how to get the `secret` portion later on:

C/C++ Rust Python C# Java

```
printf("%llx\n", (long long int)zts_node_get_id());
```

Join a network

Now we're ready to join a network:

C/C++ Rust Python C# Java

```
zts_net_join(0x1234567890abcdef);
```

The network join request will contact our network controllers (or yours!) and if the network is `public` your node will be allowed to join immediately. If however the network is `private` the node will not be allowed to join until the network owner authorizes the node.

A node is not assigned an IP address until it has successfully joined the network. This process usually takes a few seconds, thus your application must wait until the network config is received and it has been assigned an address:

C/C++ Rust Python C# Java

```
while (! zts_net_transport_is_ready(net_id)) {
  zts_util_delay(1000);
}
```

The transport readiness check above essentially just makes sure we have been authorized on the network and that we also have an assigned address and a managed route.

Getting your IP address

NOTE

Currently, only one address of each family type may be assigned (per network) to a libzt node at any time. That means one `IPv4` and one `IPv6` address. This is a technical limitation that will be removed in future versions.

C/C++ Rust Python C# Java

```
char ipstr[ZTS_IP_MAX_STR_LEN] = { 0 };
zts_addr_get_str(net_id, ZTS_AF_INET, ipstr, ZTS_IP_MAX_STR_LEN);
printf("%s\n", ipstr);
```

From another machine (or the same machine, whatever you're into), use our regular client zerotier-one (or another libzt instance) to join the same network and ping the address displayed by your new node above. You should see some jittery initial responses as our transport-triggered link is established and then once a VL2 P2P link is established the response time will stabilize.

🎉 Congratulations you've just done something cool, I guess. Ready for something with sockets?

Oh crap, I forgot. We need to stop the node when we're done:

C/C++ Rust Python C# Java

```
zts_node_stop();
```

Full example source code can be found here: libzt/examples

Sockets

TIP

If you're new to socket programming I *highly* recommend at least perusing Beej's Guide to Network Programming. This is the best reference material you'll find on the subject. It greatly helped in my own personal understanding. A portion of our C API is merely redirected calls to lwIP's C API and is thus directly compatible with what you'll learn in his guide.

Before we move onto the next section we need to quickly discuss how the socket API is structured. For each supported language ZeroTier provides a socket interface that *attempts* to be as idiomatic as possible. If you are using one of these non-C bindings the following text about the `C API` may not apply to you, but it's still good to know. I don't care about this

ZeroTier sockets can be controlled via `zts_bsd_` functions that operate nearly identically to normal BSD-style sockets, and `zts_` functions that provide simplified arguments and reduce the need to use things like `struct sockaddr`. These functions can be used on the same socket interchangeably without issue.

For instance:

- `zts_bsd_connect()` will behave in the same way as an ordinary `connect()` call (but over ZeroTier of course)
- `zts_connect()` will perform a little magic behind the scenes to deal with transport-triggered link provisioning. (i.e. re-attempting for you)
- `zts_tcp_client()` will wrap all of the common socket calls (including a `zts_connect()`) into a single call.

You are allowed to use each type of call and mix their usage among sockets as you please, as long as what you're doing makes sense at the protocol level. For instance the following is legal:

```
int sock = zts_bsd_socket(ZTS_AF_INET, ZTS_SOCK_STREAM, 0);
zts_connect(sock, addr, addrlen);
zts_bsd_write(sock, buf, buflen);
```

Client and Server (Part 2)

In this section we will:

- Use most of the same setup code as the Pingable Node example
- Set up a TCP server and client
- Send a short message between the two

To see full source code of the following with proper error and exception handling, see libzt/examples and libzt/test.

Let's start our node:

C/C++ Rust Python C# Java

```
zts_init_from_storage(storage_path);

zts_node_start();

while (! zts_node_is_online()) {
  zts_util_delay(1000);
}

printf("Node ID: %llx\n", zts_node_get_id());

zts_net_join(net_id);

while (! zts_net_transport_is_ready(net_id)) {
  zts_util_delay(1000);
}

char ipstr[ZTS_IP_MAX_STR_LEN] = { 0 };
zts_addr_get_str(net_id, family, ipstr, ZTS_IP_MAX_STR_LEN);
printf("IP address: %s\n", net_id, ipstr);
```

Notice that in the above code we use a new type of initialization function. In this case we are using a location on storage to retrieve and store our identities. We do this since we don't want to generate a unique identity for each application run. It's computationally-expensive and wasteful, think of the tardigrades.

TCP Server

Below is a simple blocking server that will open a listening socket, wait for a message and print what it receives. To see full source code of the following with proper error and exception handling, see libzt/examples and libzt/test.

C/C++ Rust Python C# Java

```
char remote_addr[ZTS_INET6_ADDRSTRLEN] = { 0 };
int remote_port = 0;
int len = ZTS_INET6_ADDRSTRLEN;

// Set up listen socket
// Note: We could also use standard zts_bsd_socket, zts_bsd_bind, etc
int fd = zts_tcp_server(local_addr, 8000, remote_addr, len, &remote_port)
printf("Accepted connection from %s:%d\n", remote_addr, remote_port);

// RX
char recvBuf[128] = { 0 };
zts_read(fd, recvBuf, sizeof(recvBuf));
printf("RX: %s\n", recvBuf);

// Cleanup
zts_close(fd);
zts_node_stop();
```

TCP Client

Below is a simple client that will connect to a remote host and send a message. To see full source code of the following with proper error and exception handling, see libzt/examples and libzt/test.

C/C++ Rust Python C# Java

```
char* msgStr = (char*)"Hello, network!";
int bytes = 0, fd;

// Set up connection socket
// Note: We could also use standard zts_bsd_socket, zts_bsd_connect, etc
fd = zts_tcp_client(remote_addr, remote_port);

// TX
zts_write(fd, msgStr, strlen(msgStr))

// Cleanup
zts_close(fd);
zts_node_stop();
```

Android Examples

In this section we will:

- Set up Android Studio projects to use a local copy of the libzt .aar
- Set up and join ZeroTier networks
- Stream video with ExoPlayer using libzt.
- Use `HttpURLConnection` to talk to an HTTP server using libzt.
- Use `OkHttp` to talk to an HTTP server using libzt.

To see full source code of the following examples, see libzt Android Examples.

Create a `zerotier.properties` file at the root of your project with the content:

```
libzt.aar=/path/to/libzt-debug.aar
```

This assumes that you have built libzt.aar locally.

Add the following to your app's build.gradle file:

```
// Creates a variable called zerotierPropertiesFile, and initializes it t
// zerotier.properties file.
def zerotierPropertiesFile = rootProject.file("zerotier.properties")

// Initializes a new Properties() object called zerotierProperties.
def zerotierProperties = new Properties()

// Loads the zerotier.properties file into the zerotierProperties object.
zerotierProperties.load(new FileInputStream(zerotierPropertiesFile))

android {
  ...
}

dependencies {
  ...
  implementation files(zerotierProperties['libzt.aar'])
}
```

Setup your network and join:

```
Context ctxt = this.getApplicationContext();

String storagePath = ctxt.getFilesDir().getAbsolutePath();

long nwid = Long.parseLong("0123456789abcdef", 16);
String remoteAddr = "10.147.19.37";
int port = 8090;


// ZeroTier setup

ZeroTierNode node = new ZeroTierNode();
node.initFromStorage(storagePath);
node.start();

Log.d(TAG, "Waiting for node to come online...");
while (! node.isOnline()) {
    ZeroTierNative.zts_util_delay(50);
}
Log.d(TAG, "Node ID: " + String.format("%010x", node.getId()));
Log.d(TAG, "Joining network...");
node.join(nwid);
Log.d(TAG, "Waiting for network...");
while (! node.isNetworkTransportReady(nwid)) {
    ZeroTierNative.zts_util_delay(50);
}
Log.d(TAG, "Joined");
```

NOTE

This is demo code and waiting on the main thread is not recommended.

ExoPlayer client

Below is a simple example to stream video with ExoPlayer using libzt.

```
// Socket logic

OkHttpClient client = new OkHttpClient.Builder()
  .socketFactory(new ZeroTierSocketsSocketFactory(remoteAddr, port))
  .build();

assert client instanceof Call.Factory;
Call.Factory callFactory = (Call.Factory) client;

OkHttpDataSource.Factory okhttpDataSourceFactory =
  new OkHttpDataSource.Factory(callFactory);

DefaultDataSource.Factory dataSourceFactory =
  new DefaultDataSource.Factory(ctxt, okhttpDataSourceFactory);

ExoPlayer player = new ExoPlayer.Builder(ctxt)
  .setMediaSourceFactory(
    new DefaultMediaSourceFactory(ctxt).setDataSourceFa
  .build();
```

For the server:

```
sudo apt install vlc
```

```
vlc mst3k.mp4 --loop --sout="#std{access=http, mux=ts, dst=:8090/sample}"
```

HttpURLConnection client

Below is an example of using `HttpURLConnection` to talk to an HTTP server using libzt.

```
// Socket logic

val url = URL(remoteUrl)
val urlConnection: HttpURLConnection = url.openConnection() as HttpURLConn


// prevent NetworkOnMainThreadException in demo code
StrictMode.setThreadPolicy(ThreadPolicy.LAX)


try{
    urlConnection.requestMethod = "GET"
    val responseCode: Int = urlConnection.responseCode

    Log.d(TAG, "response: $responseCode")
    if (responseCode == HttpURLConnection.HTTP_OK) {
        val inn = BufferedReader(InputStreamReader(urlConnection.inputStr
        var inputLine: String?
        val response = StringBuffer()
        while (inn.readLine().also { inputLine = it } != null) {
            response.append(inputLine)
        }
        inn.close()

        // print result
        println(response.toString())
    }
}
finally {
    urlConnection.disconnect()
}
```

OkHttp client

Below is an example to use `OkHttp` to talk to an HTTP server using libzt.

```
// Socket logic
val plain: MediaType = parse.parse("text/plain; charset=utf-8")
val client: OkHttpClient = Builder()
    .socketFactory(ZeroTierSocketsSocketFactory(remoteAddr, port))
    .build()
val postBody = "Hello from OkHttp!"
val request: Request = Builder()
    .url("http://www.example.com")
    .post(RequestBody.create(plain, postBody))
    .build()
client.newCall(request).enqueue(object : Callback {
    override fun onFailure(call: Call, e: IOException) {
        Log.d(TAG, "FAILURE!!")
        Log.d(TAG, e.message, e)
    }

    @Throws(IOException::class)
    override fun onResponse(call: Call, response: Response) {
        Log.d(TAG, "RESPONSE!!")
        Log.d(TAG, response.toString())
    }
})
```

Questions

I get this error when building:

```
A problem occurred evaluating project ':app'.
> exoplayerclient/zerotier.properties (No such file or directory)
```

You must create a zerotier.properties file at the root of the project with this content:

```
libzt.aar=/path/to/libzt.aar
```

I get this error when building:

```
error: cannot find symbol
import com.zerotier.sockets.ZeroTierNative;
```

The path to the libzt AAR in zerotier.properties is not correct.

Identities

Every node needs an identity, it's a cryptographic address that you'll use when authorizing the node on your private network. Don't worry though, you'll still use an IP address to talk to your node once joined to a network. You have a few mutually exclusive options to choose from in order to generate or load an identity:

C/C++ Rust Python C# Java

```
// Generate a new one
zts_init_new_identity();
// (or) Load from storage (will generate new identity if path is empty)
zts_init_from_storage("./node_path");
// (or) Load from memory
zts_init_from_memory(key, len);
```

CAUTION

You should keep your identity files safe and unique! If anyone gets a copy of these files they can impersonate your node. And if you run two nodes with the same identity simultaneously you will have a bad day.

Events

ZeroTier can send your application notifications when certain internal states are reached. Examples of this may be when the node comes online or when a new IP address is assigned by the network controller. This is set up during the initialization phase before starting your node:

C/C++ Rust Python C# Java

```
void on_zts_event(void* msgPtr)
{
  zts_event_msg_t* msg = (zts_event_msg_t*)msgPtr;
  if (msg->event_code == ZTS_EVENT_NODE_ONLINE) {
    printf("Node ID is %llx\n", msg->node->node_id);
  }
}

int main() {
  zts_init_set_event_handler(&on_zt_event);
  // ...
  zts_node_start();
  // ...
}
```

C API

In each message a pointer to a `zts_event_msg_t` object is provided. This structure contains pointers to yet more structures. Each event type has its own special structure with additional relevant data included. For instance, a network event will have a valid pointer set to `msg->network` that contains things like the network ID, its type, status, assigned addresses, multicast subscriptions, etc.

This additional information is not available in all language bindings and you may need to use the sequential-style functions to get the information you need.

Errors

On success a function typically will return `ZTS_ERR_OK` or a positive value (possibly indicating the number of bytes sent or received.) **On failure** a function typically will return a negative number or possibly a `NULL` pointer, though this may vary somewhat depending on the language binding you're using.

```
ZTS_ERR_OK        =  0, // No error
ZTS_ERR_SOCKET    = -1, // Socket error, see zts_errno
ZTS_ERR_SERVICE   = -2, // The node service experienced a problem
ZTS_ERR_ARG       = -3, // Invalid argument
ZTS_ERR_NO_RESULT = -4, // No result (not necessarily an error)
ZTS_ERR_GENERAL   = -5  // Consider filing a bug report
```

In the event of a failure from a socket call `zts_errno` will be set to a value that offers additional context. These values are defined as `zts_errno_t` in include/ZeroTierSockets.h. This is accessible via the following:

C/C++ Rust Python C# Java

```
zts_errno
```

Central API

This section will show you how to use the built-in Central API to make calls to our hosted endpoints using your language of choice. It is not necessary to use these built-in features but they are provided as a convenience.

BETA

The Central API built into libzt is still `beta` and is not included in most builds by default. It is currently only available in `C` and can be enabled by commenting out `#define ZTS_DISABLE_CENTRAL_API 1` in `ZeroTierSockets.h`. You will need your system's edition of the `libcurl` development headers and libraries.

You can read more about the Central API spec to learn more about the capabilities is exposes.

Common issues

- If you have started a node but have not received a `ZTS_EVENT_NODE_ONLINE`:
  - You may need to view our Router Config Tips knowledge base article. Sometimes this is due to firewall/NAT settings.
- If you have received a `ZTS_EVENT_NODE_ONLINE` event and attempted to join a network but do not see your node ID in the network panel on my.zerotier.com after some time:
  - You may have typed in your network ID incorrectly.
  - Used an improper integer representation for your network ID (e.g. `int` instead of `uint64_t`).
- If you are unable to reliably connect to peers:
  - Review the knowledge base article Router Config Tips. Sometimes this can be a transport-triggered link issue, and sometimes it can be a firewall/NAT issue.

Debugging

If you're experiencing odd behavior or something that looks like a bug I would suggest first reading and understanding the following sections:

- Common issues
- Thread model

If the information in those sections hasn't helped, there are a couple of ways to get debug traces out of various parts of the library.

1. Build the library in debug mode with `./build.sh host "debug"`. This will prevent the stripping of debug symbols from the library and will enable basic output traces from libzt.
2. If you believe your problem is in the network stack you can manually enable debug traces for individual modules in `src/lwipopts.h`. Toggle the `*_DEBUG` types from `LWIP_DBG_OFF` to `LWIP_DBG_ON`. And then rebuild. This will come with a significant performance cost.
3. Viewing network stack statistics: See: examples/c/statistics.c
4. There are a series of additional events which can signal whether the network stack or its virtual network interfaces have been set up properly. See `ZTS_EVENT_STACK_*` and `ZTS_EVENT_NETIF_*`.

Self-hosting

We expend considerable effort designing and maintaining a robust and globally available constellation of root servers and redundant network controllers but we understand that security practices may require you to function independently from our infrastructure. For this reason we try to make it as easy as possible to set up your own infrastructure: See here to learn more about how to set up your own network controller, and here to learn more about setting up your own roots.

Technical notes

Thread model

Both the **socket** and **control** interfaces are thread-safe but are implemented differently. The socket interface is implemented using a relatively performant core locking mechanism in lwIP. This can be disabled if you know what you're doing. The control interface is implemented by a single coarse-grained lock. This lock is not a performance bottleneck since it only applies to functions that manipulate the ZeroTier service and are called infrequently. Callback events are generated by a separate thread and are independent from the rest of the API's internal locking mechanism. Not returning from a callback event won't impact the rest of the API but it will prevent your application from receiving future events so it is in your application's best interest to perform as little work as possible in the callback function and promptly return control back to ZeroTier.

NOTE

Internally, `libzt` will spawn a number of threads for various purposes: a thread for the core service, a thread for the network stack, a low priority thread to process callback events, and a thread for each network joined. The vast majority of work is performed by the core service and stack threads.

Versioning

INFO

Each component of a libzt distribution uses semantic versioning 2.0.0.

The ZeroTier core, the libzt service, and any included language wrapper each have their own semantic versions that are loosely-coupled to each other in only one direction. libzt's `major` and `minor` version is only updated to match ZeroTier when the core version referenced via submodule is updated. Likewise, each language wrapper's version is only updated to match libzt when the version of libzt changes. Note, however that the version of libzt can change even when its core ZeroTier version does not and the same applies to libzt's wrappers! This versioning scheme is only enforced from `1.4.0` onward.

For instance:

- When released, libzt `1.4.0` will be based on ZeroTier `1.4.X`, all language wrappers would also be brought to version `1.4.0`. If a language wrapper requires an update to fix some issue we may bump the wrapper version to `1.4.1` but both libzt and ZeroTier will remain at their respective current versions. Likewise for the dependency between libzt and ZeroTier. This method was chosen over using version suffixes like `1.4.0-r1` because some semantic versioning clients may handle that version as a pre-release and prefer to use `1.4.0`.

Tags: api developer tutorial

Copyright © 2025 ZeroTier, Inc. All rights reserved.